RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Lab Evaluation 1 Report

-

"Compiler Design"

Submitted by

Md Naim Parvez
Roll No: 2003015
Section: A
4th Year (Odd Semester)

Submitted to

Farjana Parvin
Assistant Professor
Department of CSE
Rajshahi University of Engineering & Technology

Course Code: 4102

Course Title: Compiler Design Sessional

Contents

1 Introduction

2 Objective

The objective of this laboratory exercise is to understand the compilation process in C programming, specifically focusing on generating object files using the GNU Compiler Collection (GCC). This experiment demonstrates the use of Makefiles for automated compilation and the generation of object files with the extension `.o`.

3 Theory

3.1 Compilation Process

The C compilation process consists of four main stages:

1. **Preprocessing:** Handles directives like `#include` and `#define`
2. **Compilation:** Converts preprocessed code to assembly language
3. **Assembly:** Converts assembly code to machine code (object file)
4. **Linking:** Links object files with libraries to create executable

3.2 Object Files

An object file (`.o` extension) contains machine code but is not yet executable. It requires linking with system libraries and other object files to create a final executable program. The `-c` flag in GCC instructs the compiler to stop after the assembly stage and produce an object file.

3.3 Makefile

A Makefile is a build automation tool that contains rules for compiling and linking programs. It helps manage dependencies and automates the build process, making it easier to compile complex projects with multiple source files.

4 Problem Statement

Given a simple C program that calculates the sum of two integers, generate an object file named `question1.objectfile.o` using a Makefile. The Makefile should contain only the necessary commands for compilation without any extra or redundant commands.

5 Source Code

5.1 C Source File (question1.c)

```c
#include <stdio.h>
int main() {
    int num1 = 5;
    int num2 = 10;
    int sum = num1 + num2;
    printf("The sum is: %d\n", sum);
    return 0;
}
```

Listing 1: question1.c - Simple Addition Program

5.2 Makefile

```
all:
	gcc -c question1.c -o question1.objectfile.o
```

Listing 2: Makefile for Object File Generation

6 Methodology

6.1 Steps Performed

1. Created the source file `question1.c` with the given C program
2. Created a `Makefile` with a single compilation rule
3. Executed the command `make` or `make all` in the terminal
4. Verified the generation of `question1.objectfile.o`
5. Examined the object file properties and contents

6.2 Compilation Command Explanation

The GCC command used in the Makefile:

`gcc -c question1.c -o question1.objectfile.o`

Command breakdown:

- `gcc`: The GNU C Compiler
- `-c`: Compile and assemble, but do not link (generates object file)
- `question1.c`: Input source file
- `-o`: Specify output file name
- `question1.objectfile.o`: Output object file name

7 Execution and Results

7.1 Compilation Output

When executing `make` command in the terminal:

```
$ make
gcc -c question1.c -o question1.objectfile.o
$
```

The compilation completed successfully without any errors or warnings.

7.2 Generated Object File

After successful compilation, the following file was generated:

- **Filename:** question1.objectfile.o
- **File Type:** ELF 64-bit relocatable object file (on Linux systems)
- **File Size:** Approximately 1.5-2 KB (varies by system)

7.3 Verification Commands

To verify the object file generation and examine its properties:

```
# List the generated object file
$ ls -lh question1.objectfile.o
-rw-r--r-- 1 user user 1.6K Dec 12 12:00 question1.objectfile.o

# Check file type
$ file question1.objectfile.o
question1.objectfile.o: ELF 64-bit LSB relocatable, x86-64

# Display object file symbols
$ nm question1.objectfile.o
0000000000000000 T main
                 U printf
```

Listing 3: Verification Commands

7.4 Program Execution

To create an executable from the object file and run the program:

```
$ gcc question1.objectfile.o -o program
$ ./program
The sum is: 15
```

The program successfully calculated the sum of 5 and 10, displaying the output: `The sum is: 15`

8 Analysis and Discussion

8.1 Makefile Analysis

The provided Makefile is minimal and contains only the necessary command to generate the object file. It follows the requirement of having no extra or redundant commands:

- Single target: `all`
- Single compilation command using GCC
- Uses the `-c` flag to generate object file only
- Specifies custom output filename with `.objectfile.o` extension

8.2 Object File Contents

The generated object file contains:

- Machine code for the `main()` function
- Symbol table with defined and undefined symbols
- Relocation information for the `printf()` function
- Section headers for code, data, and debugging information

8.3 Key Observations

1. The object file is **not executable** on its own
2. It contains unresolved references (like `printf`) that need linking
3. The file is platform-specific (architecture and OS dependent)
4. The `-c` flag prevents automatic linking, creating only the object file

9 Advantages of Using Makefiles

- **Automation:** Simplifies the build process with a single command
- **Consistency:** Ensures the same compilation flags are used every time
- **Efficiency:** In larger projects, Make only recompiles changed files
- **Maintainability:** Centralized build configuration
- **Portability:** Same Makefile can work across different systems

10 Possible Errors and Troubleshooting

- **Syntax Errors:** Missing semicolons or braces in C code
- **Makefile Tab Issues:** Makefiles require actual tab characters, not spaces

- **File Not Found:** Ensure question1.c exists in the same directory
- **Permission Issues:** May need write permissions in the directory
- **Compiler Not Found:** GCC must be installed on the system

11 Conclusion

This laboratory exercise successfully demonstrated the generation of object files using the GCC compiler and Makefiles. The experiment covered:

- Understanding the compilation process and the role of object files
- Creating a minimal Makefile for automated compilation
- Using the `-c` flag to generate object files without linking
- Verifying the generated object file and understanding its properties
- Recognizing that object files are intermediate files in the compilation process

The Makefile contained only the necessary command to compile the source file into an object file, adhering to the requirement of avoiding extra or redundant commands. The generated `question1.objectfile.o` file was successfully created and later linked to produce a working executable that correctly calculated and displayed the sum of two integers.

12 Learning Outcomes

1. Understood the distinction between compilation and linking stages
2. Learned to use GCC flags effectively, particularly the `-c` flag
3. Gained practical experience with Makefile creation and usage
4. Understood the structure and purpose of object files
5. Recognized the importance of build automation in software development